

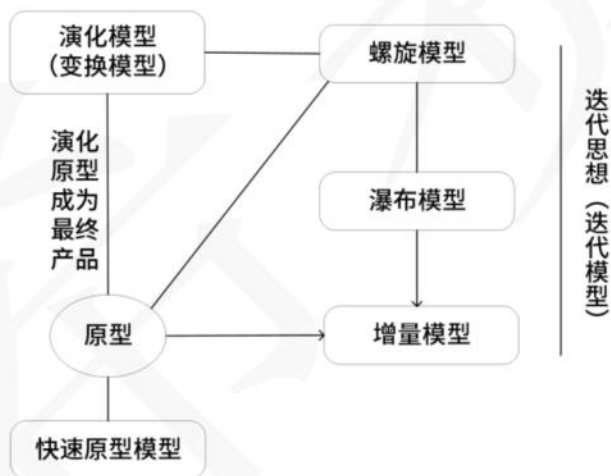
第一章 软件工程

1 考情分析

序号	知识领域	知识点详情
1	软件过程模型（映映映映）	软件过程模型
1	基于构件的软件工程（映映）	基于构件的软件工程
1	需求工程（映映）	需求工程
1	系统设计（映映）	人机界面设计（映）
2		结构化设计（映）
3		面向对象设计（映映映映映）
1	软件测试（映映映映映）	软件测试

2 考点精讲

2.1 软件过程模型（映映映映）



(1) 原型模型

典型的原型开发方法模型。适用于**需求不明确**的场景，可以帮助用户明确需求。可以分为【**抛弃型原型**】与【**演化型原型**】

原型模型两个阶段：

- 1、原型开发阶段；
- 2、目标软件开发阶段。

(2) 瀑布模型

瀑布模型是将软件生存周期中的各个活动规定为以线性顺序连接的若干阶段的模型，包括需求分析、设计、编码、运行与维护。

瀑布模型的特点是严格区分阶段，每个阶段因果关系紧密相连，只适合需求明确的项目。

缺点：软件需求完整性、正确性难确定；

严格串行化，很长时间才能看到结果；

瀑布模型要求每个阶段一次性完全解决该阶段工作，这不现实。

(3) 增量模型

融合了瀑布模型的基本成分和原型实现的迭代特征，可以有多个可用版本的发布，核心功能往往最先完成，在此基础上，每轮迭代会有新的增量发布，核心功能可以得到充分测试。强调每一个增量均发布一个可操作的产品。

(4) 螺旋模型

以快速原型为基础+瀑布模型，典型特点是引入了风险分析。它是由制定计划、风险分析、实施工程、客户评估这一循环组成的，它最初从概念项目开始第一个螺旋。

(5) V 模型

强调测试贯穿项目始终，而不是集中在测试阶段。是一种测试的开发模型。

(6) 喷泉模型

典型的面向对象的模型。特点是迭代、无间隙。会将软件开发划分为多个阶段，但各个阶段无明显界限，并且可以迭代交叉。

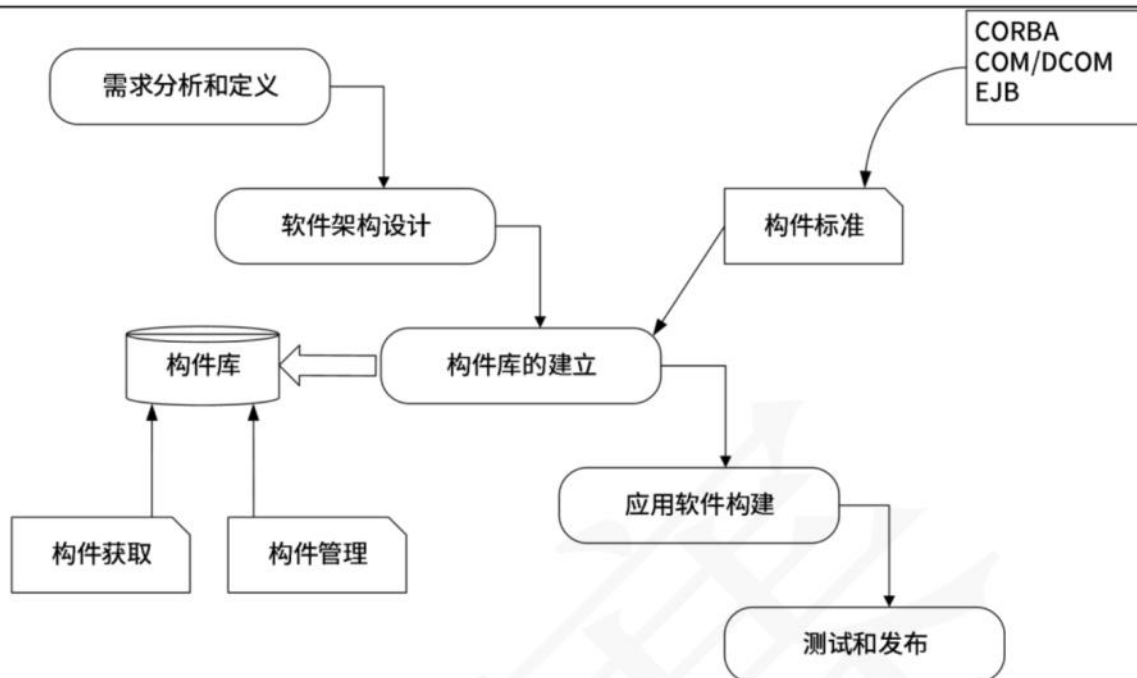
(7) 快速应用开发 RAD

概念：RAD 是瀑布模型的一个高速变种，适用比传统生命周期快得多的开发方法，它强调极短的开发周期，通常适用基于构件的开发方法获得快速开发。

过程：业务建模→数据建模→过程建模→应用生成→测试与交付

适用性：RAD 对模块化要求比较高，如果某项功能不能被模块化，则其构件就会出问题；如果高性能是一个指标，且必须通过调整结构使其适应系统构件才能获得，则 RAD 也有可能不能奏效；RAD 要求开发者和客户必须在很短的时间完成一系列的需求分析，任何一方配合不当都会导致失败；RAD 只能用于管理信息系统的开发，不适合技术风险很高的情况。

(8) 构件组装模型



【优点】易扩展、易重用、降低成本、安排任务更灵活。

【缺点】构件设计要求经验丰富的架构师、设计不好的构件难重用、强调重用可能牺牲其它指标（如性能）、第三方构件质量难控制。

(9) 统一过程（在软考中 UP、RUP 都指统一过程）

典型特点是用例驱动、以架构为中心、迭代和增量。

统一过程把一个项目分为四个不同的阶段：

构思阶段（初始/初启阶段）：定义最终产品视图和业务模型、确定系统范围。

细化阶段（精化阶段）：设计确定系统架构、制定工作计划及资源要求。

构造阶段：开发剩余构件和应用程序功能，把这些构件集成为产品，并进行详细测试。

移交阶段：确保软件对最终用户是可用的，进行β测试，制作产品发布版本。

9 个核心工作流：业务建模、需求、分析与设计、实现、测试、部署、配置与变更管理、项目管理、环境。

(10) 敏捷开发

敏捷开发是一种以人为核心、迭代、循序渐进的开发方法，适用于小团队和小项目，具有小步快跑的思想。常见的敏捷开发方法有极限编程法、水晶法、并列争球法和自适应软件开发方法。

极限编程 (XP)：一些对费用控制严格的公司中的使用，非常有效，近螺旋式的开发方法。**四大价值观**（沟通【加强面对面沟通】、简单【不过度设计】、反馈【及时反馈】、勇气【接受变更的勇气】），**十二大最佳实践**（简单设计、测试驱动、代码重构、结对编程、持续集成、现场客户、发行版本小型化、系统隐喻、代码集体所有制、规划策略、规范代码、40 小时工作机制）。

水晶方法：提倡“机动性”的方法，拥有对不同类型项目非常有效的敏捷过程。

开放式源码：程序开发人员在地域上分布很广【其他方法强调集中办公】。

SCRUM: 明确定义了可重复的方法过程。

特征驱动开发方法 (FDD): 认为有效的软件开发需要 3 要素【人、过程、技术】。定义了 6 种关键的项目角色: **项目经理、首席架构设计师、开发经理、主程序员、程序员和领域专家。**

ASD 方法: 其核心是三个非线性的、重叠的开发阶段: 猜测、合作与学习。

动态系统开发方法 (DSDM): 倡导以业务为核心。

敏捷宣言:

- 个体和交互胜过过程和工具;
- 可工作的软件胜过大量的文档;
- 客户合作胜过合同谈判;
- 响应变化胜过遵循计划。

2.2 基于构件的软件工程 (CBSE) (映映)

CBSE 体现了“购买而不是重新构造”的哲学。

CBSE 的构件应该具备的特征:

- 1、**可组装性**: 所有外部交互必须通过公开定义的接口进行。
- 2、**可部署性**: 构件总是二进制形式的, 能作为一个独立实体在平台上运行。
- 3、**文档化**: 用户根据文档来判断构件是否满足需求。
- 4、**独立性**: 可以在无其他特殊构件的情况下进行组装和部署。
- 5、**标准化**: 符合某种标准化的构件模型。

【构件的组装】:

- 1、**顺序组装**: 按顺序调用已经存在的构件, 可以用两个已经存在的构件来创建一个新的构件。
- 2、**层次组装**: 被调用构件的“提供”接口必须和调用构件的“请求”接口兼容。
- 3、**叠加组装**: 多个构件合并形成新构件, 新构件整合原构件的功能, 对外提供新的接口。

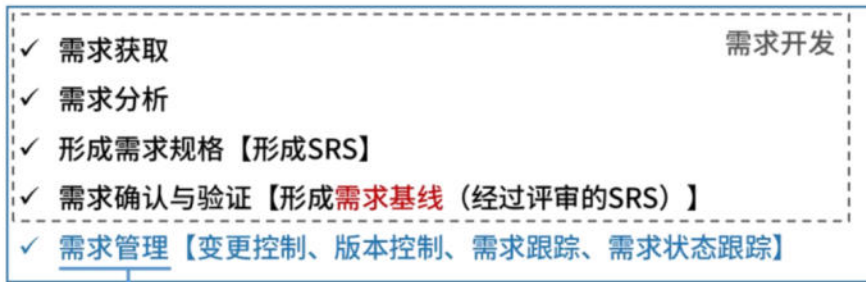
2.3 需求工程 (映映)

2.3.1 需求工程阶段划分

软件需求是指用户对系统在功能、行为、性能、设计约束等方面的期望。

软件需求是指用户解决问题或达到目标所需的条件或能力, 是系统或系统部件要满足合同、标准、规范或其他正式规定文档所需具有的条件或能力, 以及反映这些条件或能力的文档说明。

【需求工程主要活动的阶段划分】



需求管理是对【需求基线】进行管理

2.3.2 需求开发

2.3.2.1 需求获取

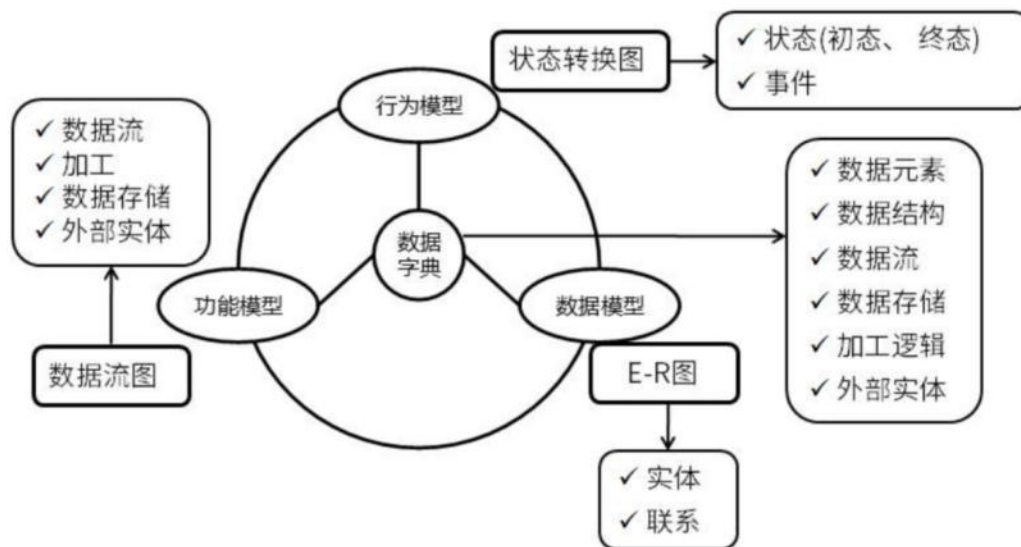


方法	特点
用户面谈	1 对 1-3，有代表性的用户，了解主观想法，交互好。成本高，要有领域知识支撑。
联合需求计划 (JRP)	高度组织的群体会议，各方参与，了解想法，消除分歧，交互好，成本高。
问卷调查	用户多，无法一一访谈，成本低。
现场观察	针对较为复杂的流程和操作。
原型化方法	通过简易系统方式解决早期需求不确定问题。
头脑风暴法	一群人围绕新业务，发散思维，不断产生新的观点。

2.3.2.2 需求分析

结构化需求分析 (SA) (映映)

(1) 结构化分析过程



(2) 结构化分析工具-数据流图 DFD

元素	说明	图元
数据流	由一组固定成分的数据组成，表示数据的流向。每个数据流通常有一个合适的名词，反映数据流的含义	
加工	加工描述了输入数据流到输出数据流之间的变换，也就是输入数据流做了什么处理后变成了输出数据流	
数据存储 (文件)	用来表示暂时存储的数据，每个文件都有名字。流向文件的数据流表示写文件，流出的表示读文件	
外部实体	指存在于软件系统外的人员或组织	

面向对象需求分析 (呜呜呜呜)

(1) 面向对象基本概念

对象：属性（数据）+方法（操作）+对象 ID

类（实体类/控制类/边界类）

实体类映射需求中的每个实体，实体类保存需要存储在永久存储体中的信息，例如，在线教育平台系统可以提取出学员类和课程类，它们都属于实体类。

控制类是用于控制用例工作的类，一般是由动宾结构的短语（“动词+名词”或“名词+动词”）转化来的名词，例如，用例“身份验证”可以对应于一个控制类“身份验证器”，它提供了与身份验证相关的所有操作。

边界类用于封装在用例内、外流动的信息或数据流。边界类位于系统与外界的交接处，包括所有窗体、报表、打印机和扫描仪等硬件的接口，以及与其他系统的接口。

继承与泛化：复用机制

封装：隐藏对象的属性和实现细节,仅对外公开接口。

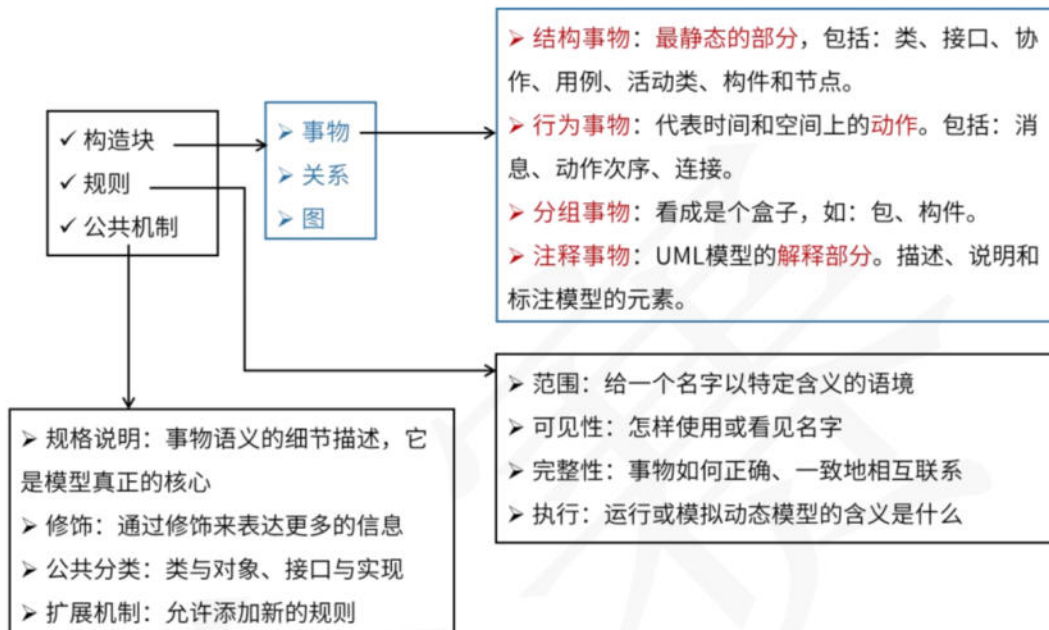
多态：不同对象收到同样的消息产生不同的结果。

接口：一种特殊的类，它只有方法定义没有实现。

重载：一个类可以有多个同名而参数类型不同的方法。

消息和消息通信：消息是异步通信的。

(2) UML 图概念



UML 包括两组公共分类，分别是类与对象（类表示概念，而对象表示具体的实体）、接口与实现（接口用来定义契约，而实现就是具体的内容）

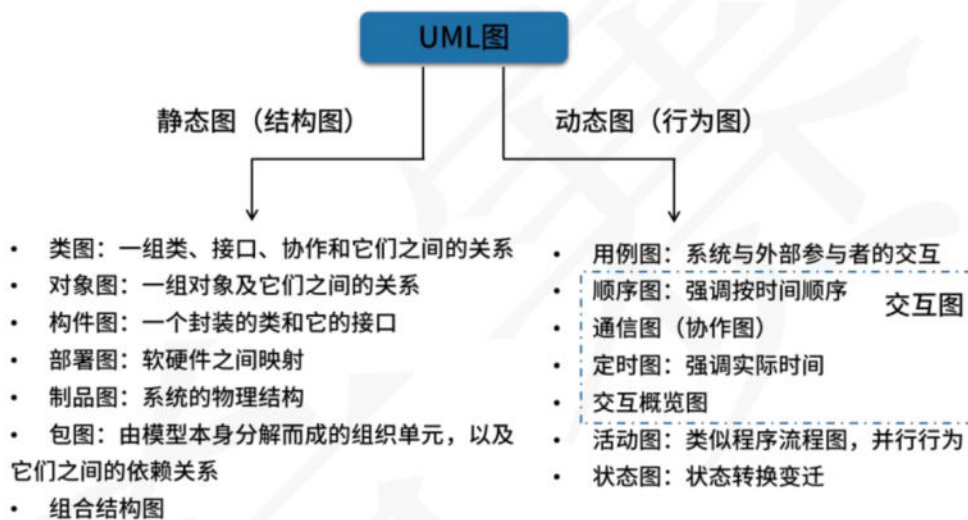
结构事物：结构事物在模型中属于最静态的部分，代表概念上或物理上的元素。UML 有七种结构事物，分别是类、接口、协作、用例、活动类、构件和节点。类是描述具有相同属性、方法、关系和语义的对象的集合，一个类实现一个或多个接口；接口是指类或构件提供特定服务的一组操作的集合，接口描述了类或构件的对外的可见的动作；协作定义了交互的操作，是一些角色和其它事物一起工作，提供一些合作的动作，这些动作比事物的总和要大；用例是描述一系列的动作，产生有价值的结果。在模型中用例通常用来组织行为事物。用例是通过协作来实现的；活动类的对象有一个或多个进程或线程。活动类和类很相似，只是它的对象代表的事物的行为和其他事物是同时存在的；构件是物理上或可替换的系统部分，它实现了一个接口集合；节点是一个物理元素，它在运行时存在，代表一个可计算的资源，通常占用一些内存和具有处理能力。一个构件集合一般来说位于一个节点，但有可能从一个节点转到另一个节点。

行为事物：行为事物是 UML 模型中的动态部分，代表时间和空间上的**动作**。UML 有两种主要的行为事物。第一种是交互（内部活动），交互是由一组对象之间在特定上下文中，为达到特定目的而进行的一系列消息交换而组成的动作。交互中组成动作的对象的每个操作都要详细列出，包括消息、动作次序（消息产生的动作）、连接（对象之间的连接）；第二种是状态机，状态机由一系列对象的状态组成。

分组事物：分组事物是 UML 模型中组织的部分，可以把它们看成是个盒子，模型可以在其中进行分解。UML 有两种分组事物，分别是包和构件。包是一种将有组织的元素分组的机制。与构件不同的是，包纯粹是一种概念上的事物，只存在于开发阶段，而构件可以存在于系统运行阶段。

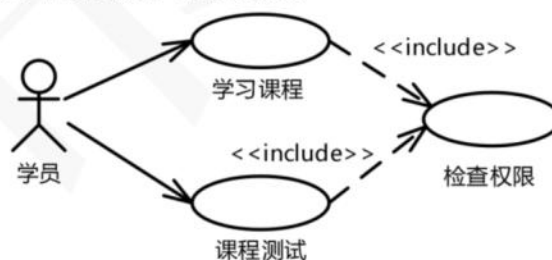
注释事物：注释事物是 UML 模型的**解释部分**。

(3) UML 图分类

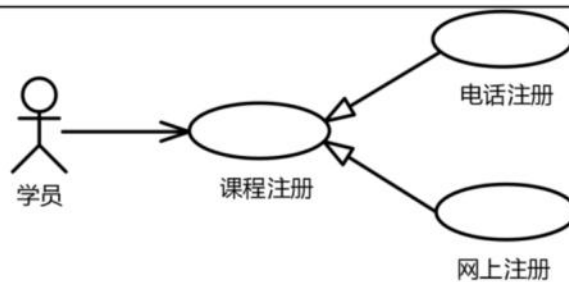


(4) UML 图关系

用例关系包括：包含关系、扩展关系、泛化关系。



图表 1 包含关系



图表 2 泛化关系



图表 3 扩展关系

包含关系: 其中这个提取出来的公共用例称为抽象用例, 而把原始用例称为基本用例或基础用例, 当可以从两个或两个以上的用例中提取公共行为时, 应该使用包含关系来表示它们。

扩展关系: 如果一个用例明显地混合了两种或两种以上的不同场景, 即根据情况可能发生多种分支, 则可以将这个用例分为一个基本用例和一个或多个扩展用例, 这样使描述可能更加清晰。

泛化关系: 当多个用例共同拥有一种类似的结构和行为的时候, 可以将它们的共性抽象成为父用例, 其他的用例作为泛化关系中的子用例。在用例的泛化关系中, 子用例是父用例的一种特殊形式, 子用例继承了父用例所有的结构、行为和关系。

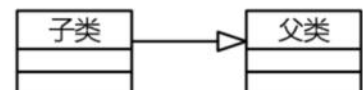
类图/对象图关系:



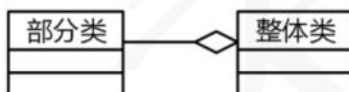
(a) 关联关系



(b) 依赖关系



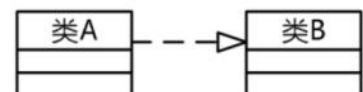
(c) 泛化关系



(d) 聚合关系



(e) 组合关系



(f) 实现关系

依赖关系: 一个事物发生变化影响另一个事物。

泛化关系: 特殊/一般关系

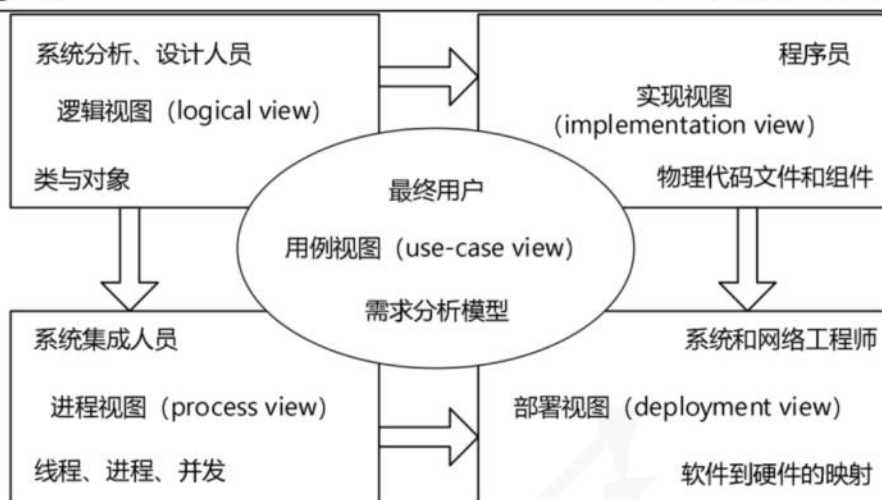
关联关系: 描述了一组链, 链是对象之间的连接。

聚合关系: 整体与部分生命周期不同。

组合关系: 整体与部分生命周期相同。

实现关系: 接口与类之间的关系

(5) “4+1” 视图 (映映)



UML 采用 4+1 视图来描述软件和软件开发过程:

逻辑视图: 以问题域的语汇组成的类和对象集合。

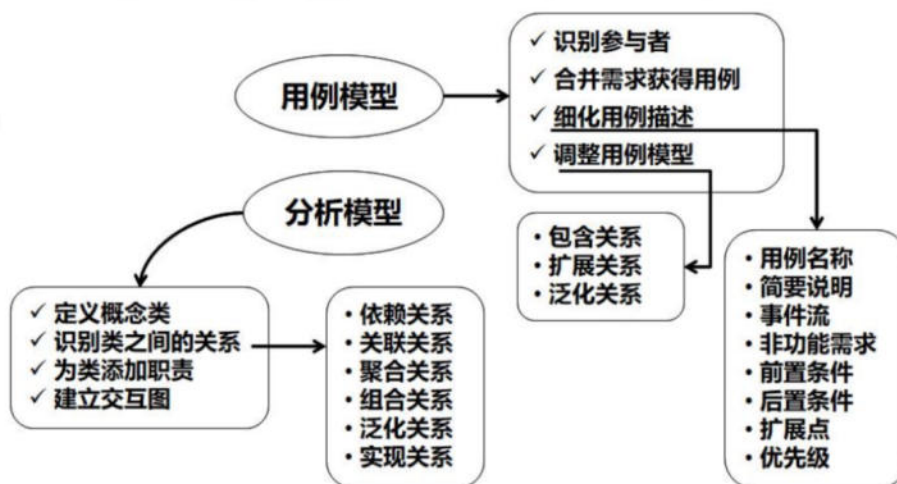
进程视图: 可执行线程和进程作为活动类的建模, 它是逻辑视图的一次执行实例, 描绘了所设计的
并发与同步结构。

实现视图: 对组成基于系统的物理代码的文件和组件进行建模。

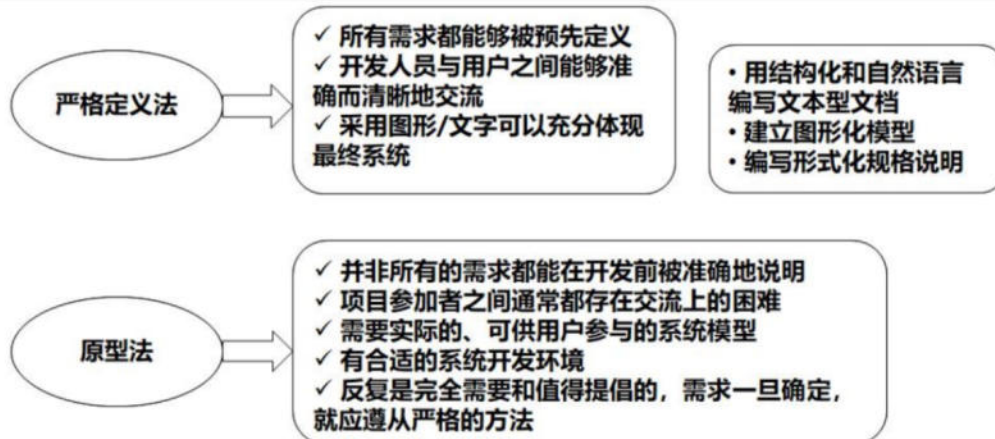
部署视图: 把构件部署到一组物理的、可计算的节点上, 表示软件到硬件的映射及分布结构。

用例视图: 最基本的需求分析模型。

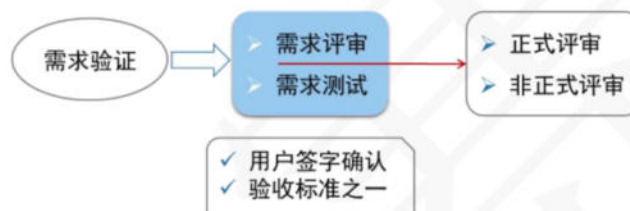
(6) OOA 需求建模



2.3.2.3 需求定义

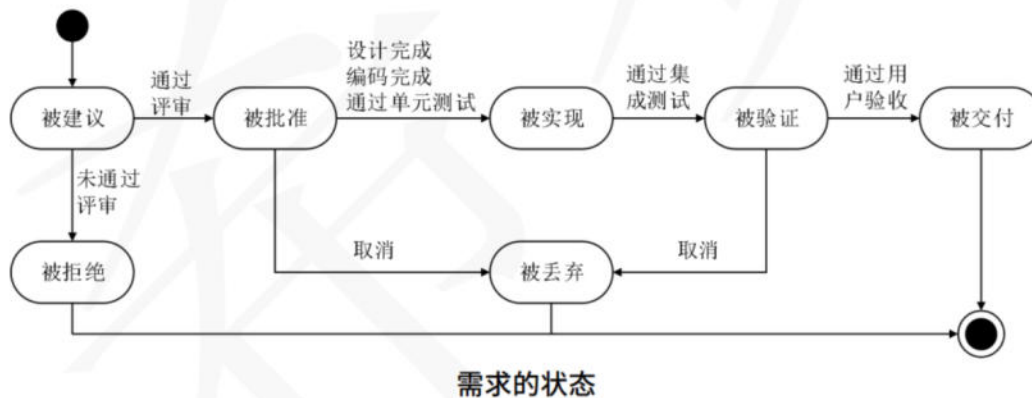


2.3.2.4 需求验证



2.3.3 需求管理 (映映映)

(1) 定义需求基线

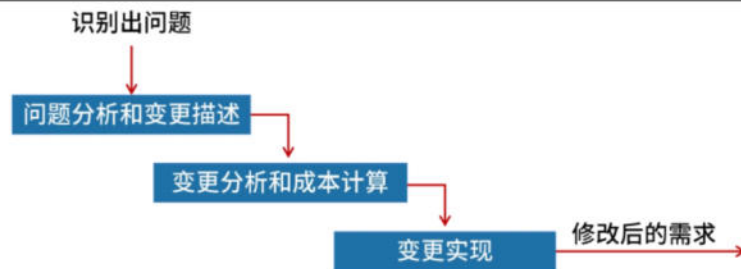


(2) 需求跟踪



(3) 变更控制

带有风险的做法：无足够用户参与，忽略了用户分类，用户需求的不断增加，模棱两可的需求，不必要的特性，过于精简的 SRS，不准确的估算。



2.4 系统设计（映映）

2.4.1 界面设计（映）

用户界面设计是指用户与系统之间架起一座桥梁，主要内容包括：定义界面形式、定义基本的交互控制形成、定义图形和符号、定义通用的功能键和组合键的含义及其操作内容、定义帮助策略等。

黄金三法则

- ★ 置于用户控制之下
- ★ 减少用户的记忆负担
- ★ 保持界面的一致性

- 以不强迫用户进入不必要的或不希望的动作的方式来定义交互方式
- 提供灵活的交互
- 允许用户交互可以被中断和撤销
- 当技能级别增加时可以使交互流水化并允许定制交互
- 使用户隔离内部技术细节
- 设计应允许用户和出现在屏幕上的对象直接交互

- 减少对短期记忆的要求
- 建立有意义的缺省
- 定义直觉性的捷径
- 界面的视觉布局应该基于真实世界的隐喻
- 以不断进展的方式揭示信息

- 允许用户将当前任务放入有意义的语境
- 在应用系列内保持一致性
- 如过去的交互模型已建立起了用户期望，除非有迫不得已的理由，不要改变它

2.4.2 结构化设计（映）

(1) 概要设计【外部设计】：功能需求分配给软件模块，确定每个模块的功能和调用关系，形成模块结构图。

(2) 详细设计【内部设计】：为每个具体任务选择适当的技术手段和处理方法。

(3) 结构化设计原则：

模块独立性原则（高内聚、低耦合）；

保持模块的大小适中；

多扇入，少扇出；

深度和宽度均不宜过高。

(4) 模块四要素：

输入和输出: 模块的输入来源和输出去向都是同一个调用者, 即一个模块从调用者那儿取得输入, 进行加工后再把输出返回调用者。

处理功能: 指模块把输入转换成输出所做的工作。

内部数据: 指仅供该模块本身引用的数据。

程序代码: 指用来实现模块功能的程序。

(5) 模块独立性的度量

1. 聚合: 衡量模块内部各元素结合的紧密程度

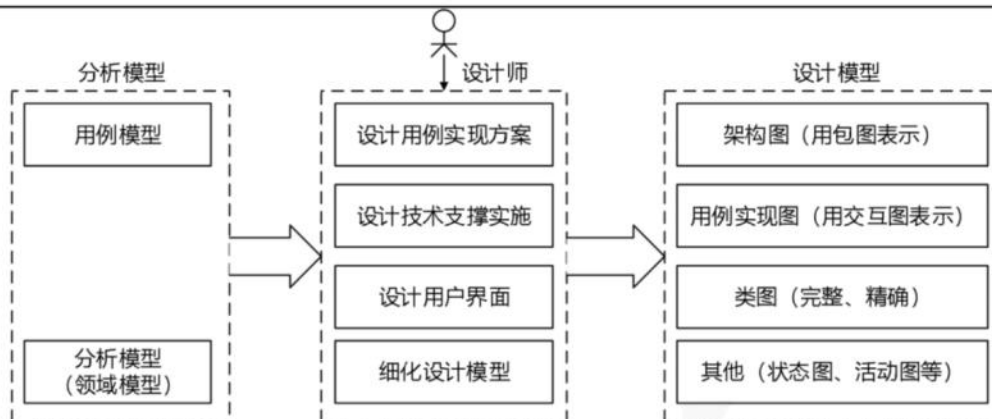
内聚类型	描述
功能内聚	完成一个 单一功能 , 各个部分协同工作, 缺一不可。
顺序内聚	处理元素相关, 而且 必须顺序执行 。
通信内聚	所有处理元素集中在一个数据结构的区域上。
过程内聚	处理元素相关, 而且必须按特定的次序执行。
时间内聚 (瞬时内聚)	所包含的任务必须在 同一时间间隔内 执行。
逻辑内聚	完成逻辑上相关的一组任务。
偶然内聚 (巧合内聚)	完成一组没有关系或松散关系的任务。

2. 耦合: 度量不同模块间互相依赖的程度

耦合类型	描述
非直接耦合	两个模块之间没有直接关系, 它们之间的联系完全是通过主模块的控制和调用来实现的。
数据耦合	一组模块借助参数表传递 简单数据 。
标记耦合	一组模块通过参数表传递记录信息 (数据结构) 。
控制耦合	模块之间传递的信息中包含用于 控制模块内部逻辑 的信息。
外部耦合	一组模块都访问同一 全局简单变量 , 而且不是通过参数表传递该全局变量的信息。
公共耦合	多个模块都访问同一个公共数据环境。
内容耦合	一个模块直接访问另一个模块的内部数据; 一个模块不通过正常入口转到另一个模块的内部; 两个模块有一部分程序代码重叠; 一个模块有多个入口。

2.4.3 面向对象设计 (映映映映映)

(1) 过程



(2) 设计原则

单一职责原则：设计目的单一的类。

开放-封闭原则：对扩展开放，对修改封闭。

李氏（Liskov）替换原则：子类可以替换父类。

依赖倒置原则：要依赖于抽象，而不是具体实现；**针对接口编程**，不要针对实现编程。

接口隔离原则：使用多个专门的接口比使用单一的总接口要好。

组合重用原则：要**尽量使用组合**，而不是继承关系达到重用目的。

迪米特（Demeter）原则（最少知识法则）：一个对象应当对其他对象有**尽可能少的了解**。

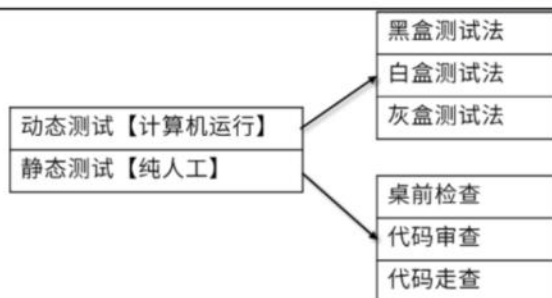
(3) 类的分类

类的分类



2.5 软件测试（快快快快快）

(1) 动态测试与静态测试



控制流分析：是否存在没有使用的语句/无法达到的语句/调用并不存在的子程序。

数据流分析：引用未定义的变量、对以前未使用的变量再次赋值。

接口分析：模块之间接口的一致性、子程序和函数之间的接口一致性、函数形参与实参的数量、顺序、类型的一致性。

表达式分析：括号不配对、数组引用越界、除数为零。

(2) 黑盒测试与白盒测试

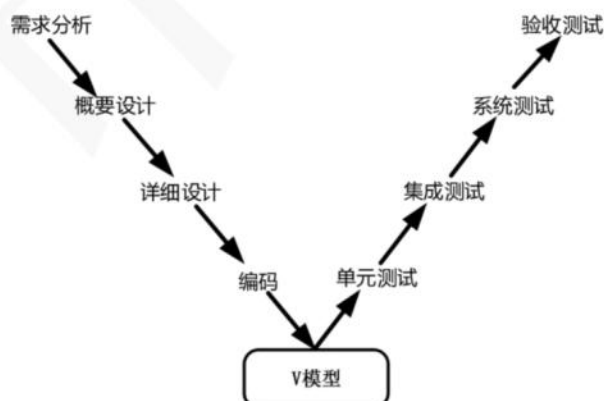
白盒测试【结构测试】：主要用于单元测试阶段。

控制流测试【逻辑覆盖测试（语句覆盖最弱，路径测试覆盖最强）】
数据流测试
程序变异测试【错误驱动测试】

黑盒测试【功能测试】：主要用于集成测试、确认测试和系统测试阶段。

等价类划分： 不同等价类，揭示不同问题；有效等价类/无效等价类。
边界值分析： $1 \leq x \leq 10$ ，可取 x 的值为 0、1、10 和 11 作为测试数据。
错误推测： 依靠测试人员的经验和直觉。
判定表： 最适合描述在多个逻辑条件取值的组合所构成的复杂情况下，分别要执行哪些不同的动作。
因果图： 根据输入条件与输出结果之间的因果关系来设计测试用例。

(3) V 模型



单元测试：依据【详细设计】，模块测试，模块功能、性能、接口等。

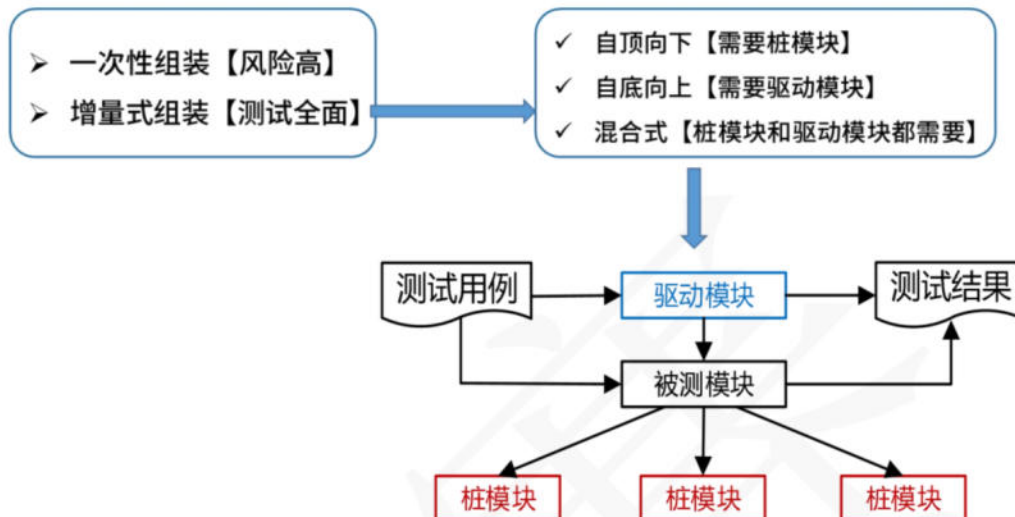
集成测试：依据【概要设计】，模块间的接口。

系统测试：依据【需求文档】，在真实环境下，验证完整的软件配置项能否和系统正确连接。

确认测试: 依据【需求文档】，验证软件与需求的一致性。内部确认测试、Alpha 测试、Beta 测试、验收测试。

回归测试: 测试软件变更之后，变更部分的正确性和对变更需求的符合性。

(4) 集成测试策略



(5) 系统测试

系统测试



第二章 信息安全技术基础知识

1 考情分析

序号	知识领域	知识点详情
1	信息加解密技术（映映映）	信息加解密技术
1	密钥管理技术（映映）	密钥管理技术
1	访问控制及数字签名技术（映映映）	访问控制及数字签名技术
1	安全架构概述（映映）	安全架构概述
1	安全模型（映映映）	安全模型
1	区块链技术（映映）	区块链技术

2 考点精讲

2.1 信息加解密技术（映映映）

2.1.1 对称加密

概念：

对称加密（又称为私人密钥加密/共享密钥加密）：加密与解密使用同一密钥。

特点：加密强度不高，但效率高；密钥分发困难。

（大量明文为了保证加密效率一般使用对称加密）

常见对称密钥加密算法：

DES：替换+移位、56 位密钥、64 位数据块、速度快、密钥易产生

3DES（三重 DES）：两个 56 位的密钥 K1、K2

加密：K1 加密->K2 解密->K1 加密

解密：K1 解密->K2 加密->K1 解密 RC-5

IDEA：128 位密钥、64 位数据块、比 DES 的加密性好、对计算机功能要求相对低，PGP。

RC-5 算法：RSA 数据安全公司的很多产品都使用了 RC-5。

AES 算法：高级加密标准，又称 Rijndael 加密法，是美国政府采用的一种区块加密标准。

2.1.2 非对称加密

概念：

非对称加密（又称为公开密钥加密）：密钥必须成对使用（公钥加密，相应的私钥解密）。

特点：加密速度慢，但强度高。密钥分发容易。

常见非对称密钥加密算法

RSA：2048 位（或 1024 位）密钥、计算量极大、难破解

ECC-椭圆曲线算法

Elgamal: 安全性依赖于计算有限域上离散对数这一难题。

2.1.3 国产密码算法

算法名称	算法特性描述	备注
SM1	对称加密, 分组长度和密钥长度都为 128 比特	广泛应用于电子政务、电子商务及国民经济的各个应用领域
SM2	非对称加密, 用于公钥加密算法、密钥交换协议、数字签名算法	国家标准推荐使用素数域 256 位椭圆曲线
SM3	杂凑算法, 杂凑值长度为 256 比特	适用于商用密码应用中的数字签名和验证
SM4	对称加密, 分组长度和密钥长度都为 128 比特	适用于无线局域网产品
SM9	标识密码算法	不需要申请数字证书, 适用于互联网应用的各种新兴应用的安全保障

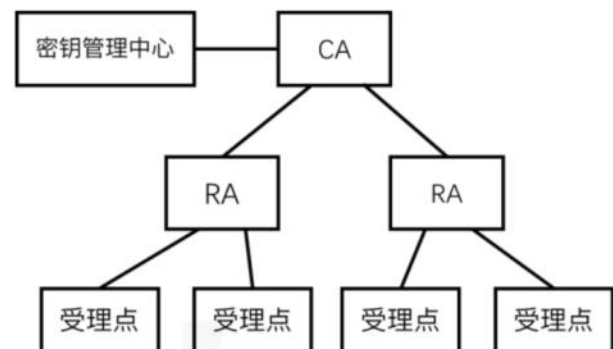
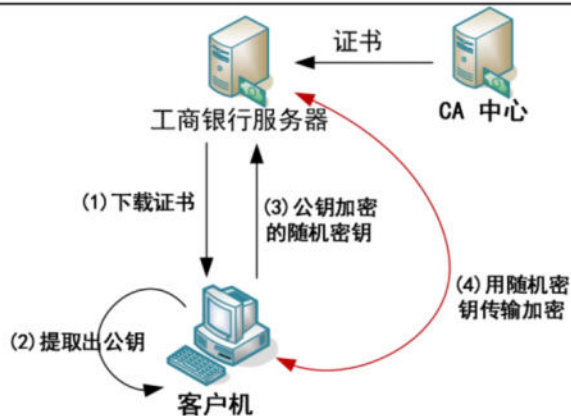
2.2 密钥管理技术 (映映)

2.2.1 数字证书

数字证书内容

- 证书的**版本信息**;
- 证书的**序列号**, 每个证书都有一个唯一的证书序列号;
- 证书所使用的**签名算法**;
- 证书的**发行机构名称**, 命名规则一般采用 X.500 格式;
- 证书的**有效期**, 现在通用的证书一般采用 UTC 时间格式, 它的计时范围为 1950-2049;
- 证书所有人的**名称**, 命名规则一般采用 X.500 格式;
- 证书所有人的**公开密钥**;
- 证书发行者对证书的**签名**。

2.2.2 公钥体系



CA（Certificate Authority）认证中心

RA（Registration Authority）注册审批机构

证书受理点

密钥管理中心-KMC

2.3 访问控制及数字签名技术（映映映）

2.3.1 访问控制技术

（1）访问控制的基本模型

访问控制的目标有两个：

- 防止非法用户进入系统；
- 阻止合法用户对系统资源的非法使用，即禁止合法用户的越权访问。

访问控制的三要素：

- **主体**：可以对其他实体的施加动作的主动实体，记为 S；
- **客体**：是接受其他实体访问的被动实体，记为 O；
- **控制策略**：是主体对客体的操作行为集和约束条件集，记为 KS。

（2）访问控制的实现技术

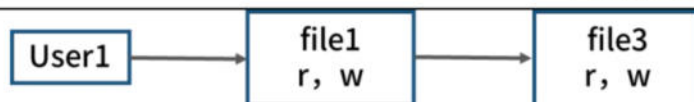
访问控制矩阵（ACM）：矩阵中的每一格表示所在行的主体对所在列的客体的访问授权。

	file1	file2	file3
User1	rw		rw
User2	r	rwX	x
User3	x	r	

访问控制列表：访问控制矩阵按列分解，具体如下：



能力表：访问控制矩阵按行分解，具体如下：



授权关系表：对应访问矩阵中每一个非空元素的实现技术。例如安全数据库系统通常用授权关系表来实现其访问控制安全机制。

2.3.2 数字签名技术（映映）

(1) 信息摘要：单向散列函数【不可逆】、固定长度的散列值。

摘要用途：确保信息【完整性】，防篡改。

常用的消息摘要算法有 MD5, SHA 等，市场上广泛使用的 MD5, SHA 算法的散列值分别为 128 和 160 位，由于 SHA 通常采用的密钥长度较长，因此安全性高于 MD5。

(2) 数字签名的过程如下图所示（发送者使用自己的私钥对摘要签名，接收者利用发送者的公钥对接收到的摘要进行验证）：



2.4 安全架构概述（映映）

(1) 信息安全面临的威胁



(2) 被动攻击：收集信息为主，破坏保密性。

攻击类型	攻击名称	描述
被动攻击	窃听（网络监听）	用各种可能的合法或非法的手段窃取系统中的信息资源和敏感信息。
	业务流分析	通过对系统进行长期监听，利用统计分析方法对诸如通信频度、通信的信息流向、通信总量的变化等参数进行研究，从而发现有价值的信息和规律。
	非法登录	有些资料将这种方式归为被动攻击方式。

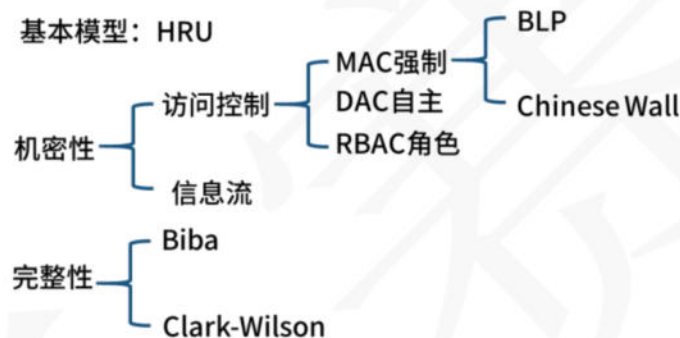
(3) 主动攻击：主动攻击的类别主要有：中断（破坏可用性），篡改（破坏完整性），伪造（破坏真实性）。

攻击类型	攻击名称	描述
主动攻击	假冒身份	非法用户冒充成为合法用户，特权小的用户冒充成为特权大的用户。
	抵赖	否认自己曾经发布过的某条消息、伪造一份对方来信等。
	旁路控制 【旁路攻击】	密码学中是指绕过对加密算法的繁琐分析，利用密码算法的硬件实现的运算中泄露的信息。如执行时间、功耗、电磁辐射等，结合统计理论快速的破解密码系统。
	重放攻击	所截获的某次合法的通信数据拷贝，出于非法的目的而被重新发送。加【时间戳】能识别并应对重放攻击。
	拒绝服务 (DOS)	破坏服务的【可用性】，对信息或其他资源的合法访问被无条件的阻止。
	XSS 跨站脚本攻击	通过利用网页【开发时留下的漏洞】，通过巧妙的方法注入恶意指令代码到网页。
攻击类型	攻击名称	描述
主动攻击	CSRF 跨站请求伪造攻击	攻击者通过一些技术手段欺骗用户的浏览器与访问一个自己曾经认证过的网站并执行一些操作（如转账或购买商品等）。

	缓冲区溢出攻击	利用【缓冲区溢出漏洞】所进行的攻击。在各种操作系统、应用软件中广泛存在。
	SQL 注入攻击	攻击者把 SQL 命令插入到 Web 表单,欺骗服务器执行恶意的 SQL 命令。 SQL 注入攻击的方式: 【恶意拼接查询】、【利用注释执行非法命令】、【传入非法参数】、【添加额外条件】。 抵御 SQL 攻击的方式包括: 【使用正则表达式】、【使用参数化的过滤性语句】、【检查用户输入的合法性】、【用户相关数据加密处理】、【存储过程来执行查询】、【使用专业的漏洞扫描工具】。

2.5 安全模型 (映映映)

分类:



(1) BLP 模型

Bell-LaPadula 模型是符合军事安全策略的计算机安全模型,简称 BLP 模型。BLP 模型的安全规则如下:

简单安全规则: 安全级别低的主体不能读安全级别高的客体;

星属性安全规则: 安全级别高的主体不能往低级别的客体写;

强星属性安全规则: 不允许对另一级别进行读写;

自主安全规则: 使用访问控制矩阵来定义说明自由存取控制。

(2) Biba 模型

BiBa 模型主要用于防止非授权修改系统信息,以保护系统的信息完整性。该模型同 BLP 模型类似,采用主体、客体、完整性级别描述安全策略要求。BiBa 模型能够防止数据从低完整性级别流向高完整性级别,其安全规则如下:

星完整性规则: 表示完整性级别低的主体不能对完整性级别高的客体写数据;

简单完整性规则: 表示完整性级别高的主体不能从完整性级别低的客体读取数据;

调用属性规则: 表示一个完整性级别低的主体不能从完整性级别高的客体调用程序或服务。

(3) Chinese Wall 模型

Chinese Wall 模型的安全策略的基础是客户访问的信息不会与当前他们可支配的信息产生冲突。

其访问客体控制的安全规则如下：

与主体曾经访问过的信息属于同一公司数据集合的信息，即墙内信息可以访问；

属于一个完全不同的利益冲突组的可以访问；

主体能够对一个客体进行写的前提是主体未对任何属于其他公司数据集进行过访问。

定理 1：一个主体一旦访问过一个客体，则该主体只能访问位于同一公司数据集的客体或在不同利益组的客体。

定理 2：在一个利益冲突组中，一个主体最多只能访问一个公司数据集。

2.6 区块链技术（块块）

(1) **【区块链】 ≠ 比特币**，比特币底层采用了区块链技术。比特币交易在我国定性为**【非法运用】**。

(2) 区块链的特点：

去中心化：由于使用分布式核算和存储，不存在中心化的硬件或管理机构，任意节点的权利和义务都是均等的，系统中的数据块由整个系统中具有维护功能的节点来共同维护。

开放性：系统是开放的，如：区块链上的**【交易信息是公开的】**，不过**【账户身份信息是高度加密的】**。

自治性：区块链采用基于协商一致的规范和协议（比如一套公开透明的算法）使得整个系统中的所有节点能够在去信任的环境自由安全的交换数据，使得对“人”的信任改成了对机器的信任，任何人为的干预不起作用。

安全性（信息不可篡改）：数据在多个节点存储了多份，篡改数据得改掉 51% 节点的数据，这太难。同时，还有其它安全机制，如：比特币的每笔交易，都由付款人用私钥签名，证明确实是他同意向某人付款，其它人无法伪造。

匿名性（去信任）：由于节点之间的交换遵循固定的算法，其数据交互是无需信任的（区块链中的程序规则会自行判断活动是否有效），因此交易对手无须通过公开身份的方式让对方自己产生信任，对信用的累积非常有帮助。

(3) **共识算法（博弈论）/全民记账**

一般有：POW（工作量证明）、PoS（权益证明）、DPoS（股份授权证明机制）

比特币采用了 POW（工作量证明）：

争夺记账权 = 挖矿

计算出来的账单节点哈希值前 13 个字符为 0，则符合规则，得到记账权。

有一个节点计算出结果，则广播消息告知其它节点，其它节点更新数据。

第三章 数据库系统

1 考情分析

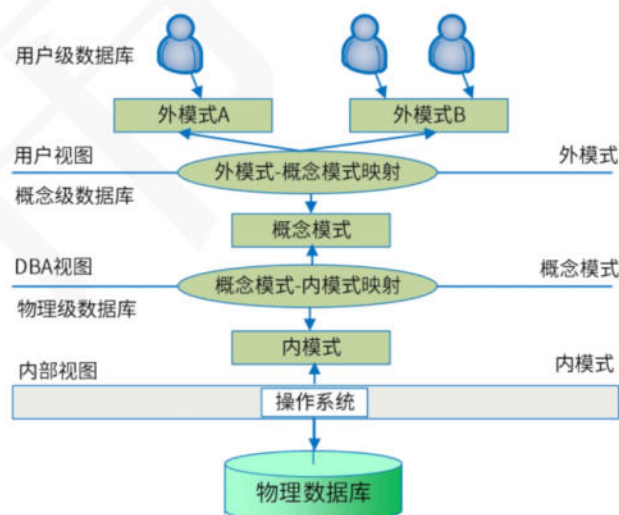
序号	知识领域	知识点详情
1	数据库概述	数据库模式、分布式数据库
1	数据库设计阶段（映映）	概念结构设计、逻辑结构设计
1	关系代数（映映映映）	关系代数
1	规范化理论（映映映映）	规范化理论

2 考点精讲

2.1 数据库概述

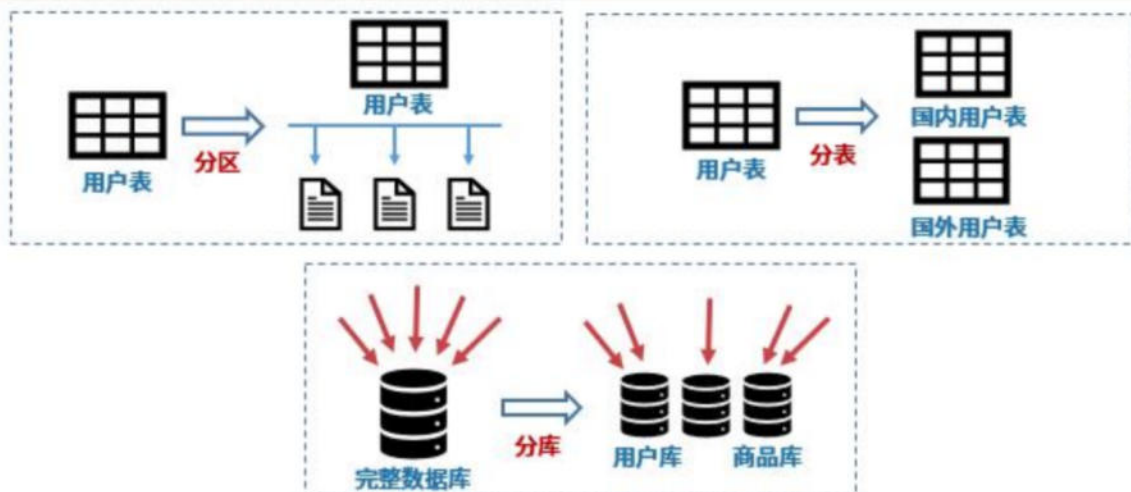
2.1.1 数据库模式

- (1) 三级模式：外模式对应视图，模式（也称为概念模式）对应数据库表，内模式对应物理文件。
- (2) 两层映像：外模式-模式映像，模式-内模式映像；两层映像可以保证数据库中的数据具有较高的逻辑独立性和物理独立性。
- (3) 物理独立性：即数据库的内模式发生改变时，应用程序不需要改变。
- (4) 逻辑独立性：即逻辑结构发生改变时，用户程序不需要改变。（逻辑独立性比物理独立性更难实现）
- (5) 聚簇索引会影响内模式



2.1.2 分布式数据库

(1) 体系结构



	分区	分表
共性	1、都针对数据表 2、都使用了分布式存储 3、都提升了查询效率 4、都降低数据库的频繁 I/O 压力值	
差异	逻辑上还是一张表	逻辑上已是多张表

分区的常见方式

分区策略	分区方式	说明
范围分区 【RANGE】	按数据范围值来做分区	例：按用户编号分区，0-999999 映射到分区 A；1000000-1999999 映射到分区 B。
散列分区 【HASH】	通过对 key 进行 hash 运算分区	例：可以把数据分配到不同分区，这类似于取余操作，余数相同的，放在一个分区上。
列表分区 【LIST】	根据某字段的某个具体值进行分区	例：长沙用户分成一个区，北京用户一个区

分区的优点：

- 1、相对于单个文件系统或是硬盘，分区可以存储更多的数据。
- 2、数据管理比较方便，比如要清理或废弃某年的数据，就可以直接删除该日期的分区数据即可。
- 3、精准定位分区查询数据，不需要全表扫描查询，大大提高数据检索效率。
- 4、可跨多个分区磁盘查询，来提高查询的吞吐量。
- 5.在涉及聚合函数查询时，可以很容易进行数据的合并。

(5) 分布式数据库管理系统-组成：LDBMS、GDBMS、全局数据字典、通信管理（CM）

(6) 分布式数据库管理系统-结构

全局控制集中的 DDBMS

全局控制分散的 DDBMS

全局控制部分分散的 DDBMS

2.1.3 索引和视图

(1) 关系的 3 种类型

基本关系（通常又称为基本表或基表）：实际存在的表，实际存储数据的逻辑表示。

查询表：查询结果对应的表。

视图表：由基表或其他视图表导出的表，本身不独立存储，数据库只存放它的定义，常称为虚表。它是一个虚拟表（逻辑上的表），其内容由查询定义（仅保存 SQL 查询语句）。同真实的表一样，视图包含一系列带有名称的列和行数据。但是，视图并没有真正存储这些数据，而是通过查询原始表动态生成所需要的数据。

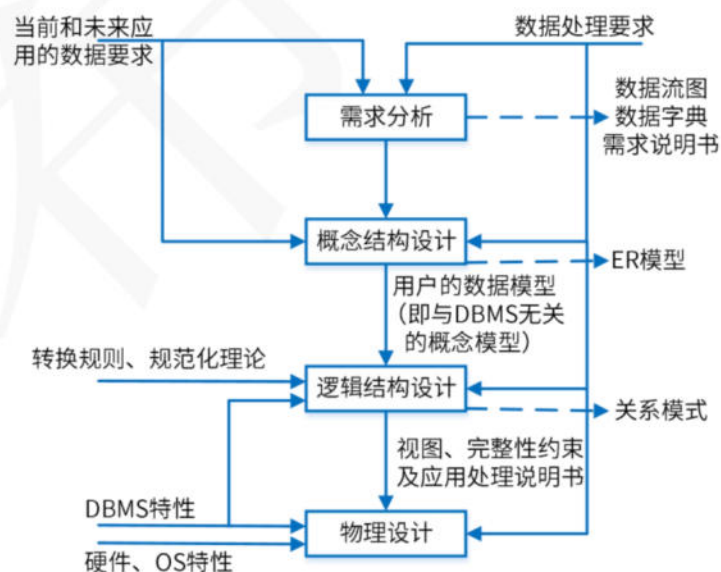
(2) 视图（View）并不在数据库中实际存在，而是一种**虚拟表**。

(3) 视图的优点：

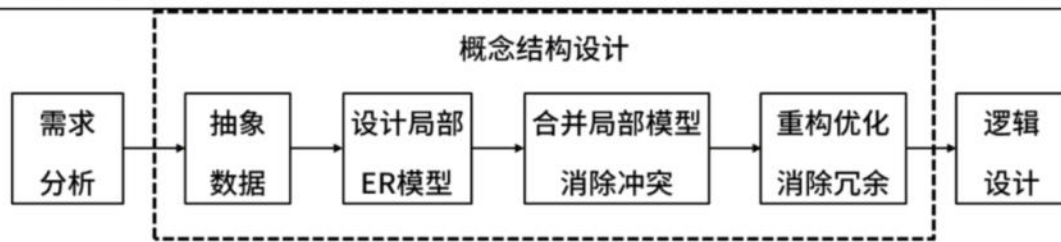
- 1、视图能**简化用户的操作**
- 2、视图机制可以使用户以不同的方式查询同一数据
- 3、视图对数据库重构提供了一定程度的**逻辑独立性**
- 4、视图可以对机密的数据提供**安全保护**

(4) **物化视图**：它不是传统意义上虚拟视图，是实体化视图，其本身会存储数据。同时当原始表中的数据更新时，物化视图也会更新。

2.2 数据库设计阶段（映映）



2.2.1 概念结构设计过程



(1) E-R 图集成的方法：

多个局部 E-R 图一次集成。

逐步集成，用累加的方式一次集成两个局部 E-R。

(2) 集成产生的冲突及解决办法：

属性冲突：包括属性域冲突和属性取值冲突。

命名冲突：包括同名异义和异名同义。

结构冲突：包括同一对象在不同应用中具有不同的抽象，以及同一实体在不同局部 E-R 图中所包含的属性个数和属性排列次序不完全相同。

2.2.2 逻辑结构设计

(1) 任务

1、E-R 图向关系模式的转换；

- 实体向关系模式的转换
- 联系向关系模式的转换

2、关系模式的规范化；

3、确定完整性约束（保证数据的正确性）；

- 实体完整性约束
- 参照完整性约束
- 用户自定义完整性约束
- 触发器

4、用户视图的确定（提高数据的安全性和独立性）。

- 根据数据流图确定处理过程使用的视图
- 根据用户类别确定不同用户使用的视图

5、应用程序设计

(2) 相关概念

目或度：关系模式中属性的个数。

候选码（候选键）

主码（主键）

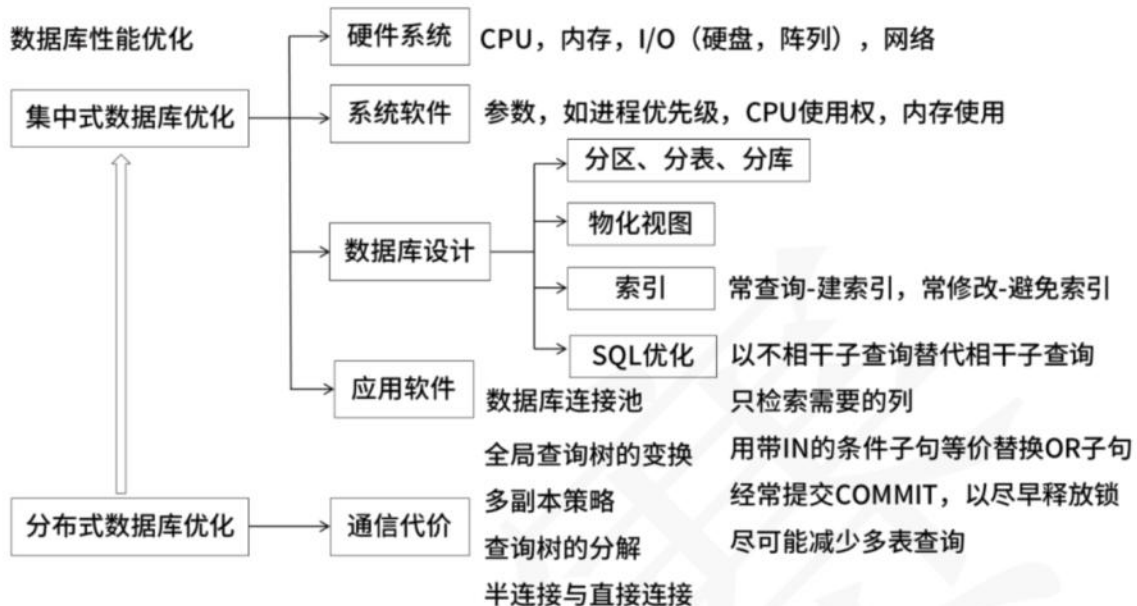
主属性与非主属性：组成候选码的属性就是主属性，其它的就是非主属性。

外码（外键）

全码 (ALL-Key) : 关系模式的所有属性组是这个关系的候选码。

简单属性与复合属性、派生属性、多值属性

2.2.3 数据库性能优化



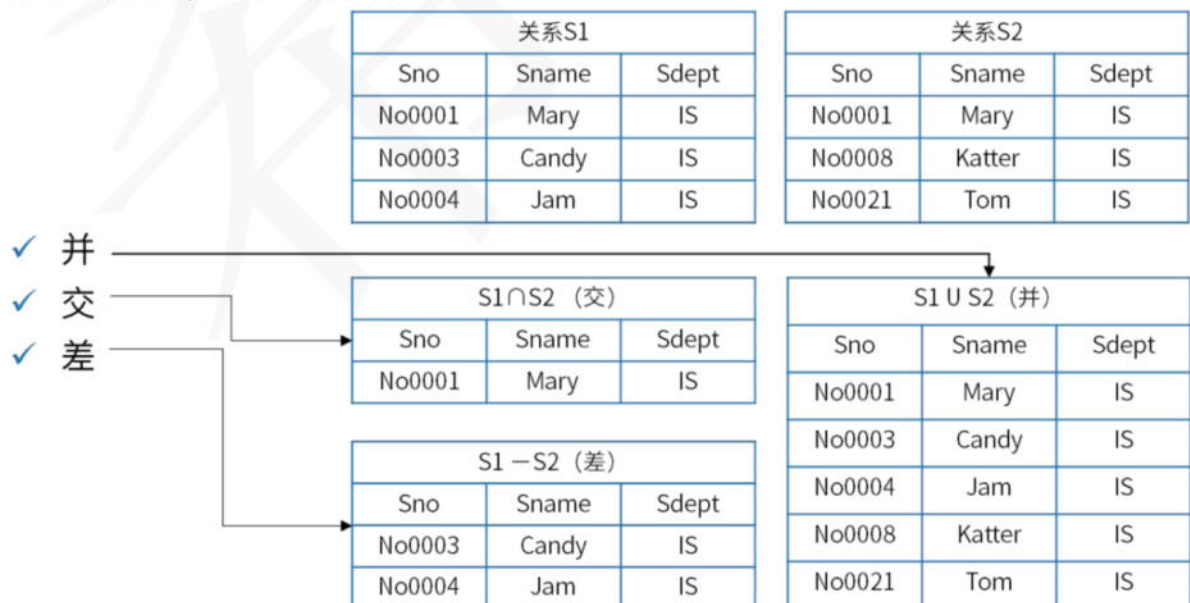
2.3 关系代数 (呲呲呲呲)

2.3.1 并 (结果为二者元组之和去除重复行)

2.3.2 交 (结果为二者重复行)

2.3.3 差 (前者去除二者重复行)

类似于集合运算, 计算如下图所示:



2.3.4 笛卡尔积

结果的属性列数为二者属性列数之和, 结果的元组行数为二者元素数乘积。

关系S1			S1 × S2 (笛卡尔积)					
Sno	Sname	Sdept	Sno	Sname	Sdept	Sno	Sname	Sdept
No0001	Mary	IS	No0001	Mary	IS	No0001	Mary	IS
No0003	Candy	IS	No0001	Mary	IS	No0008	Katter	IS
No0004	Jam	IS	No0001	Mary	IS	No0021	Tom	IS
			No0003	Candy	IS	No0001	Mary	IS
			No0003	Candy	IS	No0008	Katter	IS
			No0003	Candy	IS	No0021	Tom	IS
			No0004	Jam	IS	No0001	Mary	IS
			No0004	Jam	IS	No0008	Katter	IS
			No0004	Jam	IS	No0021	Tom	IS

2.3.5 投影

对属性列的选择列出。

2.3.6 选择

对元组行的选择列出。

属性名可以依次标序号，直接以数字形式出现在表达式中。计算如下图所示：

关系S1				投影[SELECT]: 选择列	
Sno	Sname	Sdept		选择[WHERE]: 选择行	
No0001	Mary	IS	→	$\pi_{Sno, Sname}(S1)$ (投影)	
No0003	Candy	IS		Sno	Sname
No0004	Jam	IS		No0001	Mary
				No0003	Candy
				No0004	Jam
关系S2				$\sigma_{Sno='No0003'}(S1)$ (选择)	
Sno	Sname	Sdept		Sno	Sname
No0001	Mary	IS		No0003	Candy
No0008	Katter	IS			
No0021	Tom	IS			

2.3.7 自然连接

结果列数为二者属性列数之和减去重复列,行数为二者同名属性列其值相同的结果元组。笛卡尔积、选择、投影的组合表示可以与自然连接等价。

普通连接的条件会写出,没有写出则表示为自然连接。计算如图所示：

关系S1		
Sno	Sname	Sdept
No0001	Mary	IS
No0003	Candy	IS
No0004	Jam	IS

关系S2	
Sno	Age
No0001	23
No0008	21
No0021	22



S1 ⋈ S2 (自然连接)			
Sno	Sname	Sdept	Age
No0001	Mary	IS	23

等价表示: $\pi_{1,2,3,5}(\sigma_{1=4}(S1 \times S2))$

2.4 规范化理论 (映映映映)

2.4.1 非规范化存在的问题

非规范化的关系模式, 可能存在的问题包括: 数据冗余、更新异常、插入异常、删除异常。

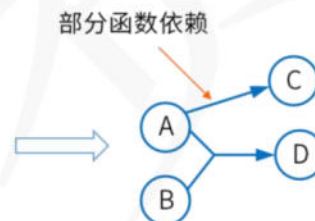
2.4.2 相关概念

函数依赖:

(1) 部分函数依赖

关系模式: R1 (A, B, C, D)

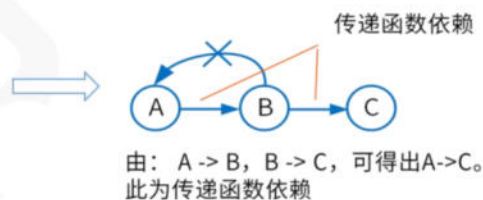
依赖集: {AB → D, A → C}



(2) 传递函数依赖

关系模式: R2 (A, B, C)

依赖集2: {A → B, B → C}



2.4.3 Armstrong 公理

对关系模式 R <U, F>来说有以下的推理规则:

自反律 (Reflexivity): 若 $Y \subseteq X \subseteq U$, 则 $X \rightarrow Y$ 成立。

增广律 (Augmentation): 若 $Z \subseteq U$ 且 $X \rightarrow Y$, 则 $XZ \rightarrow YZ$ 成立。

传递律 (Transitivity): 若 $X \rightarrow Y$ 且 $Y \rightarrow Z$, 则 $X \rightarrow Z$ 成立。

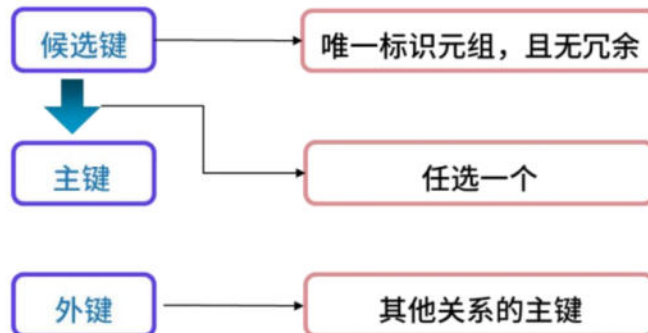
根据 A1, A2, A3 这三条推理规则可以得到下面三条推理规则:

合并规则: 由 $X \rightarrow Y$, $X \rightarrow Z$, 有 $X \rightarrow YZ$ 。(A2, A3)

伪传递规则: 由 $X \rightarrow Y$, $WY \rightarrow Z$, 有 $XW \rightarrow Z$ 。(A2, A3)

分解规则: 由 $X \rightarrow Y$ 及 $Z \subseteq Y$, 有 $X \rightarrow Z$ 。(A1, A3)

2.4.4 候选键



主属性与非主属性：组成候选码的属性就是主属性，其它的就是非主属性。

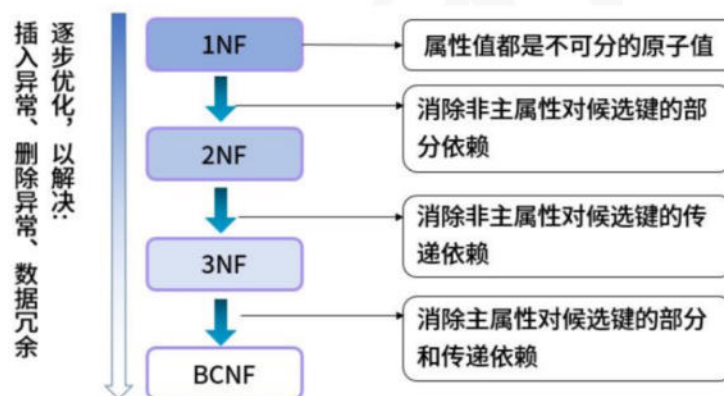
将关系模式的函数依赖关系用“有向图”的方式表示。

找入度为 0 的属性，并以该属性集合为起点，尝试遍历有向图，若能正常遍历图中所有结点，则该属性集即为关系模式的候选键。

若入度为 0 的属性集不能遍历图中所有结点，则需要尝试性的将一些中间结点（既有入度，也有出度的结点）并入入度为 0 的属性集中，直至该集合能遍历所有结点，集合为候选键。

2.4.5 范式

范式的关系



$$1NF \supset 2NF \supset 3NF \supset BCNF$$

第一范式（1NF）：在关系模式 R 中，当且仅当所有域只包含原子值，即每个属性都是不可再分的数据项，则称关系模式 R 是第一范式。

第二范式（2NF）--消除非主属性对码的部分函数依赖：当且仅当关系模式 R 是第一范式（1NF），且每一个非主属性完全依赖候选键（没有不完全依赖）时，则称关系模式 R 是第二范式。

第三范式（3NF）--消除非主属性对码的传递函数依赖：当且仅当关系模式 R 是第二范式（2NF），且 R 中没有非主属性传递依赖于候选键时，则称关系模式 R 是第三范式。

BC 范式（BCNF）--根据定义判断：设 R 是一个关系模式，F 是它的依赖集，R 属于 BCNF 当且仅当其 F 中每个依赖的决定因素必定包含 R 的某个候选码。

（可以理解为在 3NF 基础上，消除主属性之间的传递函数依赖和部分函数依赖）

2.4.6 规范化过程-模式分解

保持函数依赖分解：设数据库模式 $\rho=\{R_1, R_2, \dots, R_k\}$ 是关系模式 R 的一个分解, F 是 R 上的函数依赖集, ρ 中每个模式 R_i 上的 FD 集是 F_i 。如果 $\{F_1, F_2, \dots, F_k\}$ 与 F 是等价的（即相互逻辑蕴涵），那么称分解 ρ 保持 FD 。

无损分解：指将一个关系模式分解成若干个关系模式后, 通过自然连接和投影等运算仍能还原到原来的关系模式。

【公式法】

定理：如果 R 的分解为 $\rho=\{R_1, R_2\}$, F 为 R 所满足的函数依赖集合, 分解 ρ 具有无损连接性的充分必要条件是：

$$R_1 \cap R_2 \rightarrow (R_1 - R_2)$$

或 $R_1 \cap R_2 \rightarrow (R_2 - R_1)$

其中, $R_1 \cap R_2$ 表示模式的交, 为 R_1 与 R_2 中公共属性组成, $R_1 - R_2$ 或 $R_2 - R_1$ 表示模式的差集, $R_1 - R_2$ 表示 R_1 中去除 R_1 和 R_2 的公共属性所组成。当模式 R 分解成两个关系模式 R_1 和 R_2 时, 如果 R_1 与 R_2 的公共属性能函数决定 R_1 中或 R_2 中的其它属性, 这样的分解就具有无损连接性。

2.4.7 反规范化

(1) 反规范化手段

技术手段	说明
增加派生性冗余列	已有单价和数量列, 增加“总价”列
增加冗余列	已有学号列, 增加“姓名”列
重新组表	把拆分的表重新组表
分割表	把用户表做水平分割, 长沙的用户存在长沙, 上海的用户存在上海

(2) 反规范化的优缺点

优点：连接操作少, **检索快, 统计快**, 需要查的表减少, 检索容易。

缺点：

反规范化的缺点	解决方案
数据冗余, 需要更大存储空间	无解
插入、更新、删除操作开销更大	无解
数据不一致 可能产生添加、修改、删除异常	1、触发器数据同步 2、应用程序数据同步
更新和插入代码更难写	无解

第四章 知识产权与标准化

1 考情分析

序号	知识领域	知识点详情
1	保护范围与对象（映映映）	保护范围与对象
1	保护期限（映映）	保护期限
1	知识产权人确定（映映映）	知识产权人确定
1	侵权判断（映映映映）	侵权判断

2 考点精讲

2.1 知识产权

2.1.1 保护对象和范围（映映映）

法律法规名称	保护对象及范围	注意事项
著作权法	著作权 文学、绘画、摄影等 作品	1、不需要申请，作品完成即开始保护 2、绘画或摄影作品原件出售（赠予）著作权还归原作者， 原件拥有者有：所有权、展览权
软件著作权法 计算机软件保护条例	软件著作权 软件作品	1、不需要申请，作品完成即开始保护 2、登记制度便于举证
专利法	专利权	需要申请，专利权有效期是从申请日开始计算
商标法	商标权	需要申请，核准之日起商标受保护
反不正当竞争法	商业秘密权	1、商业秘密包括技术与经营两个方面 2、必须有保密措施才能认定商业秘密

2.1.2 保护期限（映映）

客体类型	权力类型	保护期限
公民作品	署名权、修改权、保护作品完整权	没有限制
	发表权、使用权和获得报酬权	作者终生及其死亡后的 50 年（第 50 年的 12 月 31 日）
单位作品	发表权、使用权和获得报酬权	50 年（首次发表后的第 50 年的 12 月 31 日），若期间未发表，不保护。
公民软件产品	署名权、修改权	没有限制
	发表权、复制权、发行权、出租权、信息网络传播权、翻译权、使用许可权、获得报酬权、转让权	作者终生及其死后 50 年（第 50 年的 12 月 31 日）。
单位软件产品	发表权、复制权、发行权、出租权、信息网络传播权、翻译权、使用许可权、获得报酬权、转让权	50 年（首次发表后的第 50 年的 12 月 31 日），若其期间未发表，满 50 年则不再保护
注册商标		有效期 10 年（若注册人死亡或倒闭 1 年后，未转移则可注销，期满后 6 个月内必须续注）
发明专利权		保护期为 20 年（从申请日开始）
实用新型和外观设计专利权		保护期为 10 年（从申请日开始）
商业秘密		不确定，公开后公众可用

2.1.3 知识产权人确定（映映映）

情况说明		判断说明	归属
作品	职务作品	利用单位的物质技术条件进行创作,并由单位承担责任的	除署名权外其他著作权归单位
		有合同约定,其著作权属于单位	除署名权外其他著作权归单位
		其他	作者拥有著作权,单位有权在业务范围内优先使用
软件	职务作品	属于本职工作中明确规定的开发目标	单位享有著作权
		属于从事本职工作活动的结果	单位享有著作权
		使用了单位资金、专用设备、未公开的信息等物质、技术条件,并由单位或组织承担责任的软件	单位享有著作权
专利权	职务作品	本职工作中作出的发明创造	单位享有专利
		履行本单位交付的本职工作之外的任务所作出的发明创造	单位享有专利权
		离职、退休或调动工作后 1 年内,与原单位工作相关	单位享有专利权

情况说明		判断说明	归属
作品 软件	委托	有合同约定，著作权归委托方	委托方
	创作	合同中未约定著作权归属	创作方
	合作 开发	只进行组织、提供咨询意见、物质条件或者进行其他辅助工作	不享有著作权
		共同创作的	共同享有，按人头比例。 成果可分割的，可分开申请。
商标		谁先申请谁拥有（除知名商标的非法抢注） 同时申请，则根据谁先使用（需提供证据） 无法提供证据，协商归属，无效时使用抽签（但不可不确定）	
专利		谁先申请谁拥有 同时申请则协商归属，协商不成则同时驳回双方的专利申请	

2.1.4 侵权判断(映映映映)

(1) 适用情景

中国公民、法人或者其他组织的作品,不论是否发表,都享有著作权。

开发软件所用的思想、处理过程、操作方法或者数学概念不受保护。

著作权法不适用于下列情形:

法律、法规,国家机关的决议、决定、命令和其他具有立法、行政、司法性质的文件,及其官方正式译文;

时事新闻;

历法、通用数表、通用表格和公式。

(2) 常见侵权与合理引用区分

不侵权	侵权
✓ 个人学习、研究或者欣赏; ✓ 适当引用; ✓ 公开演讲内容; ✓ 用于教学或科学研究; ✓ 复制馆藏作品; ✓ 免费表演他人作品; ✓ 室外公共场所艺术品临摹、绘画、摄影、录像; ✓ 将汉语作品译成少数民族语言作品或盲文出版。	✓ 未经许可,发表他人作品; ✓ 未经合作作者许可,与他人合作创作的作品当作自己单独创作的作品发表的; ✓ 未参加创作,在他人作品署名; ✓ 歪曲、篡改他人作品的; ✓ 剽窃他人作品的; ✓ 使用他人作品,未付报酬; ✓ 未经出版者许可,使用其出版的图书、期刊的版式设计的。

制作于 23 年 11 月 适用于第 2 版教材



免费订阅考试资讯



更多备考资料及学习福利
扫码添加希赛网课程顾问了解



免费在线刷题